RISC-V Processor Design and Verification: Single-Cycle, Pipeline, and RV32M Extension

Digital Electronics — Final Project Report

Camilla Malossi
2026 - Tongji University



1. Introduction

1.1 Project Objective

This project implements a RISC-V processor in Verilog HDL through three progressive stages of development. The first stage produces a fully functional single-cycle processor supporting the complete RV32I base integer instruction set, verified through a self-checking simulation testbench. The second stage extends the design to a classic five-stage pipelined architecture (IF, ID, EX, MEM, WB) adding full data forwarding, load-use hazard detection, and branch/jump resolution, while reusing the same test program to confirm correctness. The third stage adds the RV32M integer multiplication and division extension, contributing eight new instructions to the pipelined processor with minimal changes to the existing datapath. Each stage is verified independently through Icarus Verilog simulation.

1.2 Specifications

The processor is implemented as a 32-bit architecture with fixed-length 32-bit instruction encoding, operating on a 32-bit datapath throughout. The register file consists of 32 general-purpose registers (x0–x31), with x0 permanently hardwired to zero as mandated by the RISC-V specification. The memories are sized to accommodate the full test programs: the single-cycle design uses a 128-word instruction memory and a 256-byte data memory, while the pipelined testbench provides 256-word (1 KB) instruction and data memories. Both versions support byte, halfword and word reads and writes.

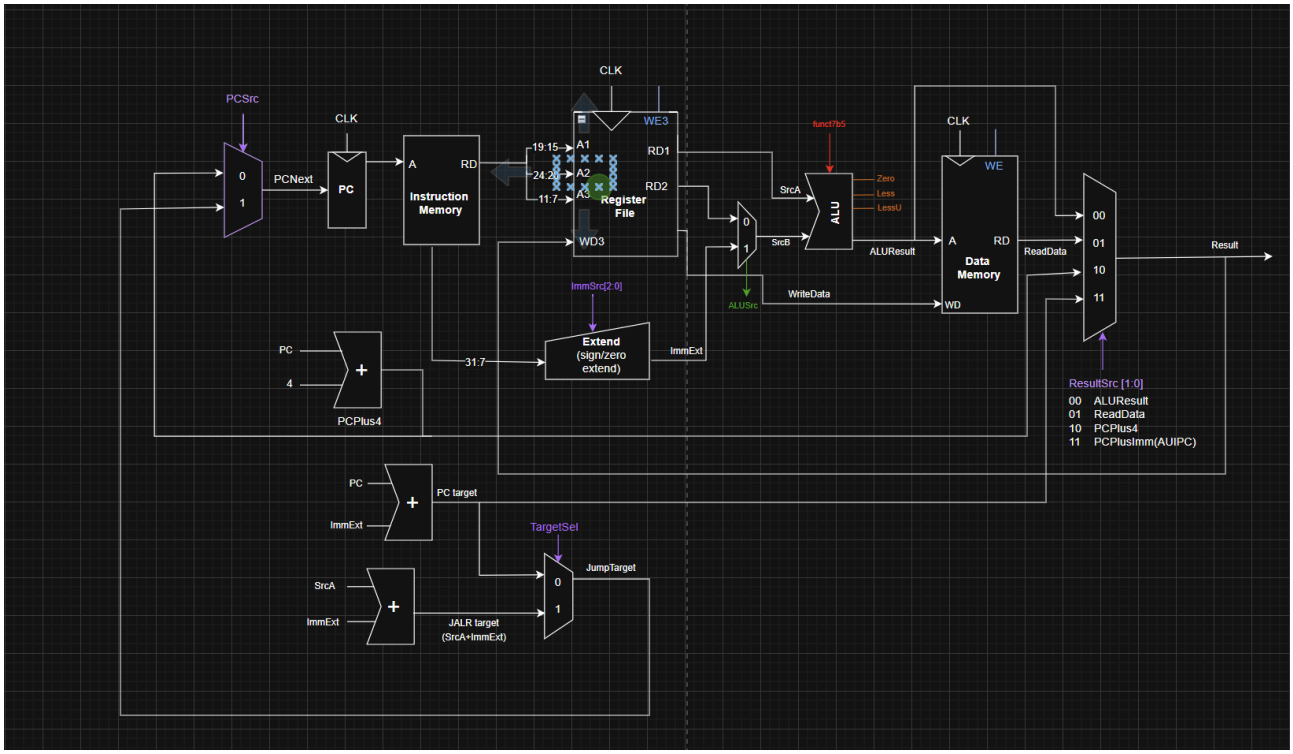
The project was developed in three successive stages. The first is a single-cycle implementation in which every instruction completes in exactly one clock cycle (CPI = 1), providing a simple and fully verifiable baseline. The second is a five-stage pipelined implementation (IF/ID/EX/MEM/WB) that improves throughput by overlapping instruction execution, with full data forwarding, load-use hazard detection, and predict-not-taken branch handling. The third adds the RV32M integer multiply and divide extension to the pipelined design. Across all three stages, the instructions FENCE, PAUSE, ECALL, and EBREAK are not implemented.

1.3 Initial architecture Choice: Single-Cycle

A single-cycle architecture was chosen as the initial implementation for its conceptual simplicity and ease of verification. Every instruction completes its entire execution within a single clock cycle, which eliminates pipeline hazards and forwarding complexity and makes it well-suited for a first complete RV32I implementation. The single-cycle design was subsequently extended to a classic five-stage pipelined architecture (IF/ID/EX/MEM/WB), described in Section 5, to improve throughput and demonstrate the trade-offs between simplicity and performance.

2. Single-Cycle processor Architecture

2.1 Block Diagram



The datapath of the processor was designed and drawn using draw.io and is shown in the figure above. It extends the base single-cycle datapath presented in Harris & Harris (*Digital Design and Computer Architecture: RISC-V Edition*) with additional hardware to support the complete RV32I instruction set, including JALR, AUIPC, and signed and unsigned branch comparisons. The entire datapath is combinational between clock edges, meaning that every instruction, regardless of its type, completes its full execution within a single clock cycle.

The datapath is organised as a left-to-right flow through five conceptual stages: fetch, decode, execute, memory access, and write-back. Although these stages are not physically separated by registers as they would be in a pipelined design, they provide a useful framework for describing the signal propagation through the circuit.

The following section walks through each functional block of the datapath in detail, describing its role in the execution of an instruction, its inputs and outputs, and how it connects to the surrounding logic.

Fetch. At the start of each cycle, the 32-bit PC register outputs the address of the current instruction. This address is fed directly to the instruction memory, which returns the 32-bit instruction word combinationally on the same clock cycle. Simultaneously, a dedicated adder computes PCPlus4 = PC + 4, representing the address of the next sequential instruction. PCPlus4 serves two purposes: it is one of the two inputs to the PCNext mux that determines the next value of the PC, and it is also routed all the way to the write-back stage where it can be written to the destination register for JAL and JALR instructions as the return address.

Decode. The 32-bit instruction word is distributed to multiple consumers in parallel. Bits [19:15] and [24:20] address the two read ports of the register file (A1 and A2), while bits [11:7] address the write port (A3). The register file responds combinationally, producing SrcA on output RD1 and WriteData on output RD2. Bits [31:7] of the instruction are simultaneously fed to the Extend block, which sign- or zero-extends the embedded immediate field into a full 32-bit value called ImmExt. The Extend block supports all five RISC-V immediate formats — I, S, B, J, and U — selecting the appropriate one via the 3-bit control signal ImmSrc[2:0], shown in purple in the diagram to indicate that it is an addition to the base Harris & Harris datapath. The opcode (bits [6:0]), funct3 (bits [14:12]), and funct7b5 (bit [30])

fields are forwarded to the controller, which is not shown in the datapath diagram but generates all control signals from them.

Execute. The ALU receives two operands. The first, SrcA, comes directly from register file output RD1. The second, SrcB, is selected by a 2-to-1 mux controlled by the ALUSrc signal, shown in green. When ALUSrc is zero, SrcB equals WriteData (RD2), which is the value of the second source register and is used by all R-type instructions. When ALUSrc is one, SrcB equals ImmExt from the Extend block, which is used by all immediate-format instructions including loads, stores, branches, and the I-type ALU operations. The ALU performs the operation specified by ALUControl and produces ALUResult as its primary output. In addition to the result, the ALU generates three condition flags that are routed to the controller: Zero, which is asserted when the result is zero and is used to evaluate BEQ and BNE; Less, a signed comparison flag computed as the XOR of the result's sign bit and the overflow bit, used for BLT, BGE, and SLT; and LessU, an unsigned comparison flag used for BLTU, BGEU, and SLTU. These three flags are shown in orange in the diagram. The ALU also receives the funct7b5 signal directly from bit [30] of the instruction, shown in red, which it uses to distinguish SUB from ADD and arithmetic right shifts (SRA, SRAI) from logical right shifts (SRL, SRLI), since both pairs share the same opcode and funct3 encoding and differ only in funct7.

Memory. ALUResult is used as the byte address for the data memory. For store instructions, the value of WriteData — which carries the contents of register rs2 — is written to that address when the WE signal is asserted by the controller. For load instructions, the data memory reads from the address and returns ReadData combinationally in the same cycle, applying the appropriate sign or zero extension at byte or halfword granularity based on funct3.

Write-back. The final stage selects the value to be written back to the destination register rd through a 4-to-1 mux controlled by the 2-bit signal ResultSrc[1:0], shown in purple. When ResultSrc is 00, the mux selects ALUResult, which is the case for all R-type and I-type ALU instructions. When it is 01, it selects ReadData from the data memory, covering all load instructions. When it is 10, it selects PCPlus4, which is written to the link register for JAL and JALR instructions so that the callee can return to the correct address. When it is 11, it selects PCPlusImm = PC + ImmExt, which is the result of the AUIPC instruction that places a PC-relative address into a register. The selected value is written to the register file at WD3 on the next rising clock edge, provided that the write-enable signal WE3 is asserted.

PC update logic. The PC update path is the most significant extension relative to the base Harris & Harris datapath and is drawn in the lower portion of the diagram. Two separate adders compute the two possible jump targets. The first, labelled PCTarget, computes PC + ImmExt and is used for PC-relative jumps and branches, including all six branch instructions and JAL. The second, labelled JALR target, computes SrcA + ImmExt, implementing the register-based absolute addressing of the JALR instruction. A 2-to-1 mux controlled by TargetSel, shown in purple, selects between these two targets to produce JumpTarget: TargetSel = 0 selects PCTarget and TargetSel = 1 selects JalrTarget. The JumpTarget signal is then routed back up to the PCNext mux at the far left of the datapath, where it competes with PCPlus4 for selection. The PCNext mux is controlled by PCSrc, also shown in purple: when PCSrc is zero the processor advances sequentially to PCPlus4, and when PCSrc is one it redirects execution to JumpTarget. The controller asserts PCSrc whenever a taken branch is detected — by evaluating the Zero, Less, or LessU flags against the branch type — or when a JAL or JALR instruction is being executed.

The colour coding used throughout the diagram serves to distinguish signals by their origin and nature. Purple signals are control inputs generated by the controller that were not present in the original Harris & Harris datapath. These are PCSrc, TargetSel, ResultSrc[1:0], and ImmSrc[2:0]. Blue signals are control inputs that were already present in the base design, namely WE3, WE, and CLK. The funct7b5 signal is shown in red to highlight its direct path from the instruction word into the ALU, bypassing the controller entirely. Orange signals are the ALU output flags Zero, Less, and LessU, which travel from the ALU to the controller to enable branch condition evaluation. The ALUSrc signal controlling the SrcB mux is shown in green.

2.2 Principal Modules

Module	Function
top	Top-level module. Instantiates and connects riscvsingle, imem, and dmem.
riscvsingle	Processor core. Instantiates controller and datapath.
controller	Control unit. Combines maindec and aludec, evaluates all six branch conditions, generates PCSrc.
maindec	Main decoder. Maps opcode → RegWrite, ImmSrc, ALUSrc, MemWrite, ResultSrc, Branch, ALUOp, Jump, JalrSrc.
aludec	ALU decoder. Maps funct3 + funct7[5] given ALUOp → ALUControl[2:0].
datapath	PC register, adders, register file, immediate extender, ALU source muxes, ALU, 4-way result mux.
alu	Implements ADD, SUB, AND, OR, XOR, SLL, SRL, SRA, SLT, SLTU. Outputs Zero, Less (signed), LessU (unsigned).
extend	Sign-extends immediates for all five RISC-V formats: I, S, B, J, U (selected by ImmSrc[2:0]).
regfile	32×32-bit register file. Synchronous write; asynchronous read. x0 always reads zero.
imem	128-word ROM, initialised from finaltest_base.txt via $readmemh. Word-addressed by PC[31:2].
dmem	256-byte RAM with byte-addressable read/write. Supports SB/SH/SW on write; LB/LH/LW/LBU/LHU on read.

2.3 Instruction Format Summary

Format	Examples	Bit-field layout
R-type	ADD, SUB, SLL…	`funct7[6:0] | rs2[4:0] | rs1[4:0] | funct3[2:0] | rd[4:0] | opcode[6:0]`
I-type	ADDI, LW, JALR	`imm[11:0] | rs1[4:0] | funct3[2:0] | rd[4:0] | opcode[6:0]`
S-type	SB, SH, SW	`imm[11:5] | rs2[4:0] | rs1[4:0] | funct3[2:0] | imm[4:0] | opcode[6:0]`
B-type	BEQ, BLT…	`imm[12|10:5] | rs2[4:0] | rs1[4:0] | funct3[2:0] | imm[4:1|11] | opcode[6:0]`
U-type	LUI, AUIPC	`imm[31:12] | rd[4:0] | opcode[6:0]`
J-type	JAL	`imm[20|10:1|11|19:12] | rd[4:0] | opcode[6:0]`

3. RV32I Implementation Details

3.1 Implemented Instructions

Type	Instructions	Key operation
R-type	`ADD, SUB, SLL, SLT, SLTU, XOR, SRL, SRA, OR, AND`	Two register operands; result to rd
I-type (ALU)	`ADDI, SLTI, SLTIU, XORI, ORI, ANDI, SLLI, SRLI, SRAI`	Register + sign-extended 12-bit immediate
I-type (load)	`LB, LH, LW, LBU, LHU`	Load from memory address rs1+imm with sign/zero extension
S-type	`SB, SH, SW`	Store rs2 to memory at rs1+imm
B-type	`BEQ, BNE, BLT, BGE, BLTU, BGEU`	PC-relative branch on condition
U-type	`LUI, AUIPC`	20-bit upper immediate into rd
J-type	`JAL, JALR`	Unconditional jump with link (rd = PC+4)

3.2 Key Implementation Details

ALU (Signed and Unsigned Comparisons)

- less `(signed)`: computed as `sum[31] XOR overflow`. Used by BLT, BGE, SLT.
- lessu `(unsigned)`: computed as `($unsigned(a) < $unsigned(b))`. Used by BLTU, BGEU, SLTU.
- zero: result == 0. Used by BEQ, BNE.

Shift Instructions

- `SLL/SLLI`: logical left shift, zeros at LSB. ALUControl = 3'b110.
- `SRL/SRLI`: logical right shift, zeros at MSB. ALUControl = 3'b111, funct7b5 = 0.
- `SRA/SRAI`: arithmetic right shift, MSB replicated. ALUControl = 3'b111, funct7b5 = 1. Implemented with `$signed(a) >>> b[4:0]`.

Load and Store (Sub-word Access)

- `SB/SH/SW`: write 1, 2, or 4 bytes respectively (little-endian for multi-byte).
- `LB/LH`: read and sign-extend byte/halfword to 32 bits.
- `LBU/LHU`: read and zero-extend byte/halfword to 32 bits.
- `LW`: read full 32-bit word (funct3 = 010, standard encoding).

dmem funct3 Mapping

The data memory funct3 encoding was corrected to match the RISC-V standard: funct3=000→LB, 001→LH, 010→LW, 100→LBU, 101→LHU. This allows all load instructions to use their standard assembler encodings.

Immediate Extension

- `I (000)`: Instr[31:20] sign-extended.
- `S (001)`: Instr[31:25] and Instr[11:7] concatenated and sign-extended.
- `B (010)`: Instr[31,7,30:25,11:8] assembled with LSB=0, sign-extended.
- `J (011)`: Instr[31,19:12,20,30:21] assembled with LSB=0, sign-extended.
- `U (100)`: Instr[31:12] in upper 20 bits, lower 12 cleared.

PC Logic and Jumps

PCNext is selected between PC+4 and a jump/branch target. The target is PC+ImmExt for branches and JAL, or rs1+ImmExt for JALR. PCSrc is asserted when BranchTaken is true or a JAL/JALR is executing. All six branch conditions are evaluated combinationally from the Zero, Less, and LessU flags.

4. Processor Verification

4.1 Methodology

The processor is verified with a self-checking Verilog testbench. The testbench monitors MemWrite on every falling clock edge and declares success when it observes a write of value 25 to address 100. A write to any unexpected address triggers an immediate failure message.

The test stimulus is a 100-instruction assembly program (finaltest_base.txt) covering 18 functional test groups. Each group begins with ADDI ra,ra,1 and ends with a conditional branch that skips an infinite-loop trap on success. Any incorrect result traps the processor permanently, preventing the final store to address 100.

The instruction memory was expanded from 64 to 128 words to accommodate the full test program. The change requires two lines in the Verilog source:

```
// In module imem:
reg [31:0] RAM [127:0];                          // was [63:0]
initial $readmemh("finaltest_base.txt", RAM, 0, 99); // was 0, 63

// In testbench:
repeat (300) @(posedge clk);  // was 150
```

4.2 Pass/Fail Counter

Register `ra` is initialised to 0 and incremented once per group. After all 18 groups, ra has been incremented 18 times. A 19th increment occurs inside G18 itself, upon return from JALR, confirming that the jump landed at the correct address. A final `ADDI ra, ra, 6` yields 25. The constant 6 equals 25 minus 19, so any extra or missing increment produces a value different from 25, making the counter self-checking.

4.3 Test Group Summary

#	Instructions	What is verified	Pass condition
G01	`LUI + AUIPC`	U-type decode, PC-relative addressing	BEQ x0,x0 (trivial) → confirms no crash
G02	`ADD`	R-type addition	t1=sp+gp=8; compared against t2=8
G03	`SUB`	R-type subtraction	t1=sp-gp=2; compared against t0=2
G04	`AND + OR + XOR`	R-type bitwise operations (all three)	BEQ x0,x0 → all three execute without crash
G05	`SLT + SLTU`	Signed and unsigned set-less-than	Both give 1 for gp<sp; results compared
G06	`SLL + SRL`	Logical shift left and right	5<<2=20>>2=5; result compared against sp=5

G07	`SRAI + SRA`	Arithmetic shift right (imm + register)	BEQ x0,x0 → execute without crash
G08	`ORI + ANDI + XORI`	I-type bitwise with immediate	BEQ x0,x0 → all three execute without crash
G09	`SLTI + SLTIU`	I-type set-less-than with immediate	Both give 1 for gp<5; results compared
G10	`BNE`	Branch not equal	sp≠gp → branch taken; trap if not
G11	`BLT + BGE`	Signed branch less-than / greater-equal then	3<5 and 5≥3; both branches must be taken
G12	`BLTU + BGEU`	Unsigned branch less-than / greater-equal then	3<(u)5 and 5≥(u)3; both branches must be taken
G13	`SB + LB`	Byte store / signed byte load	0xFF sign-extended to -1; compared against tp=-1
G14	`LBU`	Unsigned byte load	0xFF zero-extended to 255; compared against 255
G15	`SH + LH`	Halfword store / signed halfword load	0x0005 sign-extended to 5; compared against sp=5
G16	`LHU`	Unsigned halfword load	0x0005 zero-extended to 5; compared against sp=5
G17	`SW + LW`	Word store / word load	sp=5 stored and reloaded; compared against sp=5
G18	`JAL + JALR`	PC-relative and register-based jump	JALR target 0x180 verified by landing on ADDI ra,ra,1

4.4 Memory Access Summary

Group	Instr.	Addr.	Operation	Expected result
G13	`SB`	`96`	Write 0xFF (LSB of tp=-1)	RAM[96]=0xFF; simout data=4294967295
G13	`LB`	`96`	Read byte, sign-extend → -1	t1=0xFFFFFFFF=-1=tp ✓
G14	`LBU`	`96`	Read byte, zero-extend → 255	t2=0x000000FF=255 ✓
G15	`SH`	`96-97`	Write 0x0005 (LE halfword)	RAM[96]=0x05, RAM[97]=0x00; simout data=5
G15	`LH`	`96-97`	Read halfword, sign-extend → 5	t1=0x00000005=5=sp ✓
G16	`LHU`	`96-97`	Read halfword, zero-extend → 5	t1=0x00000005=5=sp ✓
G17	`SW`	`96-99`	Write word 5	RAM[96..99]=5; simout data=5
G17	`LW`	`96-99`	Read word → 5	t1=0x00000005=5=sp ✓
end	`SW`	`100`	Write word 25 → SUCCESS	RAM[100]=25; testbench SUCCESS

✓= active verification

4.5 Simulation Results

The following output was produced by Icarus Verilog with a 300-cycle testbench:

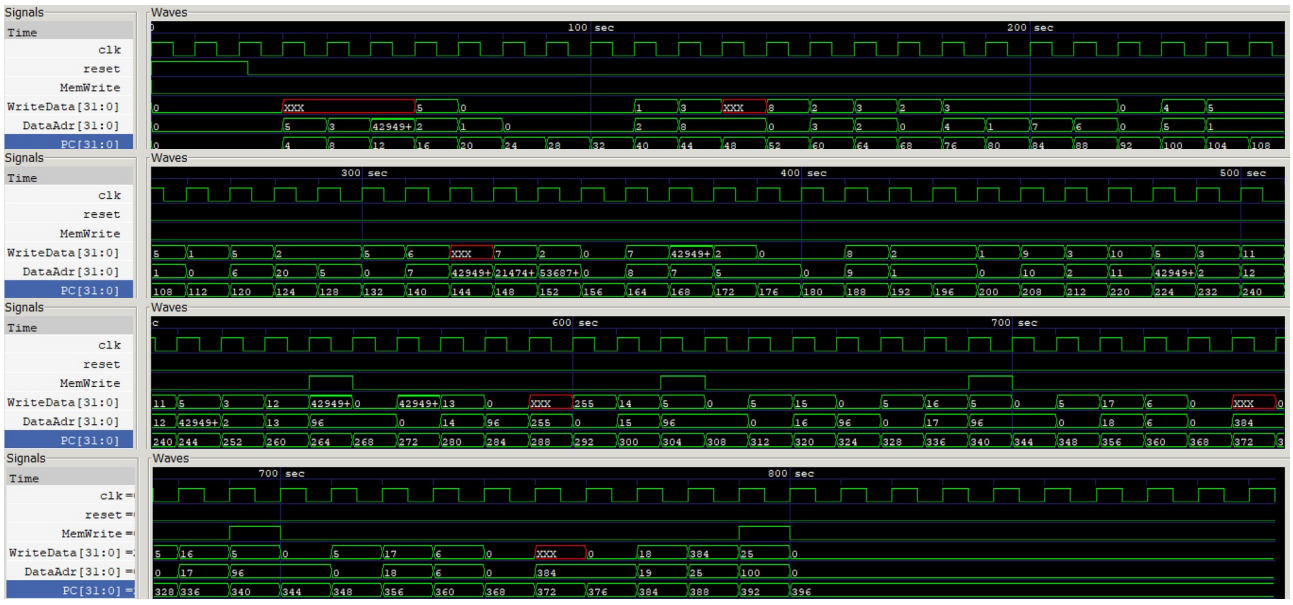
```
VCD info: dumpfile riscvsingle1.vcd opened for output.
MEMWRITE at addr=96 data=4294967295
MEMWRITE at addr=96 data=5
MEMWRITE at addr=96 data=5
MEMWRITE at addr=100 data=25
Simulation succeeded - All tests passed!
```

- addr=96, data=4294967295: G13 SB stores byte 0xFF (WriteData bus = 0xFFFFFFFF; only LSB written to RAM).
- addr=96, data=5: G15 SH stores halfword 0x0005.
- addr=96, data=5: G17 SW stores word 5, overwriting the same address.
- addr=100, data=25: Final SW stores the accumulated counter value 25. This means all 18 groups passed.

4.6 Waveform Analysis

The simulation generates riscvsingle1.vcd for GTKWave inspection. The key signals to observe are:

- MemWrite: asserted for four cycles (three writes to addr=96, one to addr=100).
- DataAdr: 96 for the first three MemWrite pulses, then 100 for the final one.
- WriteData: 4294967295, 5, 5, 25 for the four write events.
- PC: advances sequentially except during branches (which skip their traps) and JAL/JALR.

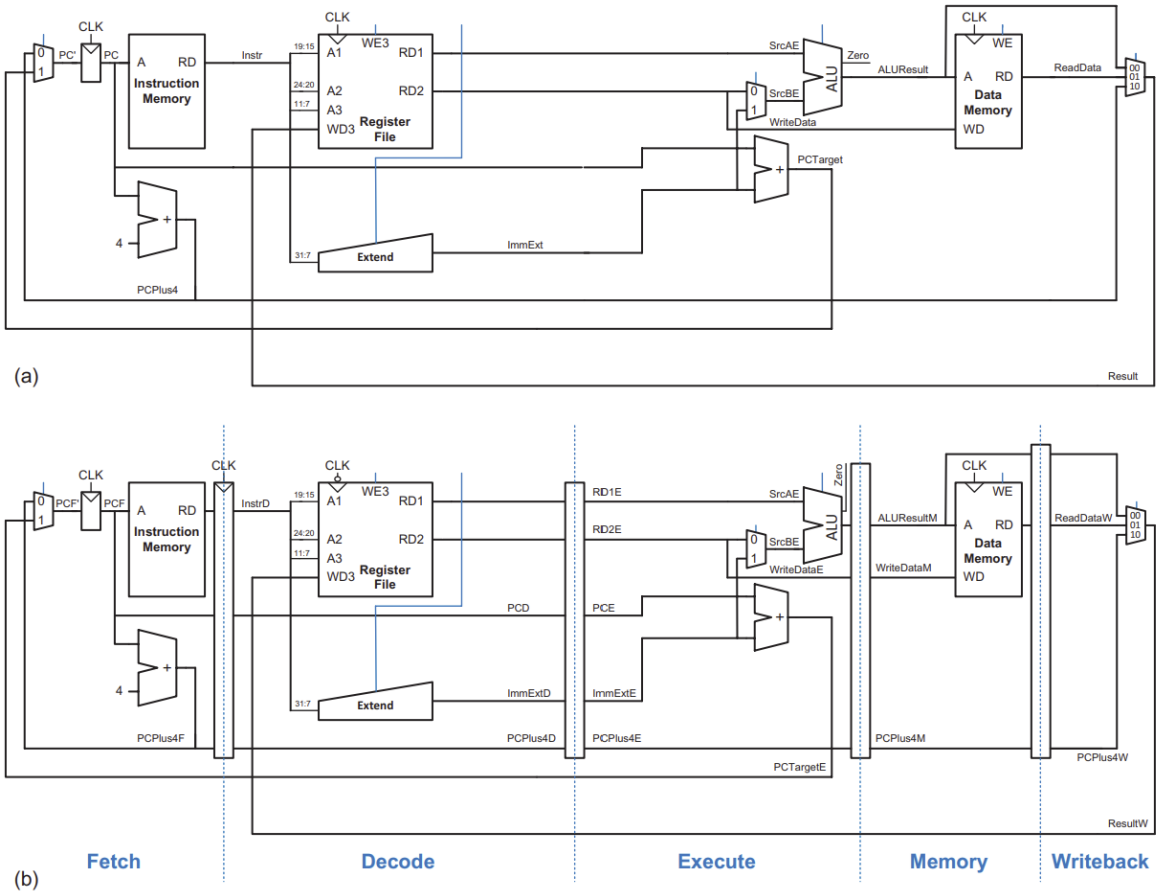


5. Pipelined Extension

5.1 Motivation and Design Goals

The single-cycle processor described in the previous sections provides a correct and verifiable baseline, but its clock period is constrained by the longest combinational path in the design which is the load instruction. This is because it has to traverse the instruction memory, register file, ALU, data memory, and write-back mux in a single cycle. A five-stage pipeline breaks this critical path into five shorter, balanced stages, each separated by a pipeline register. This, in the ideal case of perfectly balanced allows the clock frequency to increase by roughly a factor of five, raising throughput proportionally. The cost is additional control complexity: the pipeline must detect and resolve data hazards caused by instruction dependencies and control hazards caused by branches and jumps.

5.2 Datapath Analysis



5.2.1 From Single-Cycle to Pipelined: Key Structural Differences

The two diagrams illustrate the fundamental transformation from a single-cycle to a pipelined datapath. In the single-cycle design (a), every signal flows combinationally from left to right in a single clock cycle: the PC addresses the instruction memory, the register file is read, the ALU computes, the data memory is accessed if needed, and the result is written back, all within one period. The pipeline version (b) introduces four sets of registers at the stage boundaries, visible as the dashed vertical lines, that capture all signals at the end of each clock cycle and hold them for the next stage. Each stage now operates independently and in parallel with the others, so that while one instruction is in the Execute stage, the next is already being decoded and the one after that is being fetched.

The signal naming convention also changes: in the pipelined design, every signal carries a suffix indicating which stage it currently occupies. For example, the program counter appears as PCF in Fetch, PCD in Decode, PCE in Execute, and so on through to PCPlus4W at Write-back. This makes it explicit that the same logical value travels through the pipeline over multiple cycles, rather than being computed and consumed in one.

5.2.2 What the Book Diagram Does Not Show

The diagram above is from the textbook Harris &Harris and shows a simplified baseline that omits several features required by a complete RV32I implementation. The following additions were made in this project:

The first concerns jump instructions. The book pipeline handles branches but does not show the full hardware needed for JAL and JALR. JAL requires computing PC+ImmExt as the jump target directly from the EX stage PC, while JALR requires computing rs1+ImmExt and masking the least-significant bit to zero. A dedicated TargetSel mux selects between the two targets, and a link control bit propagates through the pipeline to write PC+4 to the destination register at write-back for both instructions.

The second concerns hazard handling. The book diagram shows the basic pipeline structure but does not include the hazard detection unit or the forwarding unit. The hazard detection unit monitors the EX stage for load instructions whose destination register matches a source register of the instruction in ID, and inserts a one-cycle stall when this occurs. The forwarding unit resolves the remaining data hazards by routing the most recent register value directly from the EX/MEM or MEM/WB pipeline register back to the ALU inputs, bypassing the register file entirely.

The third concerns branch resolution. The book uses a simple zero flag from the ALU to detect BEQ, but a complete RV32I implementation requires all six branch conditions (BEQ, BNE, BLT, BGE, BLTU, BGEU) evaluated on the full register values. A dedicated branch unit in the EX stage receives the forwarded operands and evaluates the appropriate condition directly, replacing the flag-based approach.

Finally, the RV32M multiply and divide extension is not part of the book datapath at all. Since all eight RV32M instructions are R-type and follow the same pipeline path as any other register-to-register operation, no structural changes to the pipeline were needed: the only modifications were widening the ALU operation code from 4 to 5 bits in the control unit and adding the eight new cases to the ALU, as described in Section 6.

5.3 Pipeline Stages

The pipeline follows the classic IF/ID/EX/MEM/WB decomposition. In the first stage, Instruction Fetch, the program counter addresses the instruction memory and advances to PC+4 on every cycle unless a branch, JAL, or JALR redirects it. The fetched instruction then enters the Instruction Decode stage, where it is decoded, the register file is read, the immediate is sign-extended, and all control signals are generated; the resulting operands and control signals are latched into the ID/EX pipeline register before proceeding. In the Execute stage the ALU computes the result or memory address, branch conditions are evaluated against the forwarded register values, and any taken branch or jump causes the pipeline to flush the two instructions already in flight and redirect the PC to the correct target. The Memory Access stage is where load and store instructions interact with the data memory, while all other instructions simply pass their ALU result through unchanged. Finally, in the Write-Back stage, the appropriate value, whether an ALU result, a loaded datum, or PC+4 for link instructions such as JAL and JALR, is written back to the destination register in the register file.

5.4 Hazard Handling

5.4.1 Data Hazards — Forwarding

A data hazard occurs when an instruction depends on the result of an instruction still in the pipeline. The pipelined processor resolves most data hazards through a full forwarding unit, which detects when the destination register of an instruction in the EX/MEM or MEM/WB stage matches a source register of the instruction currently in the EX stage, and routes the most recent value directly to the ALU inputs. Two forwarding paths are implemented. The first, EX→EX, forwards the ALU result directly from the EX/MEM pipeline register to the ALU input of the immediately following instruction. The second, MEM→EX, forwards the write-back value from the MEM/WB pipeline register and is used when the EX/MEM path is not applicable. When both paths could theoretically provide a value, priority is always given to the EX/MEM path, since it carries the more recently produced result.

5.4.2 Load-Use Hazards — Stall

Forwarding cannot resolve a load-use hazard, which arises when the instruction immediately following a load reads the register being loaded. The loaded data is not available until the end of the MEM stage, one cycle after the dependent instruction would need it in EX. The hazard detection unit identifies this condition — a load in EX with a destination register matching a source register of the instruction in ID — and inserts a one-cycle stall: the PC and IF/ID register are frozen for one cycle and a bubble (NOP) is injected into the ID/EX register.

5.4.3 Control Hazards — Branch and Jump

Branches and jumps are resolved at the end of the EX stage. Since two instructions have already been fetched by the time the outcome is known, the predict-not-taken strategy is used: the processor always fetches the next sequential instruction. If the branch is taken or a jump is executing, the two instructions in the IF and IF/ID stages are invalidated by flushing the IF/ID and ID/EX pipeline registers (inserting two bubbles), and the PC is redirected to the correct target. The penalty for a taken branch or any jump is therefore two cycles.

5.5 Differences from the Single-Cycle Implementation

The pipelined processor reuses the same instruction set, immediate generator, and ALU logic, so the core computational logic remains unchanged. The principal structural differences lie in how the datapath is organised around the pipeline. Four pipeline registers — IF/ID, ID/EX, EX/MEM, and MEM/WB — are introduced to separate and synchronise the five stages, holding all operands and control signals as they propagate through the design. LUI and AUIPC are handled slightly differently from the single-cycle version: rather than relying on a separate result mux at write-back, the first ALU operand is forced to zero or to the current PC value directly in the ID stage, before the value is latched into the ID/EX register. Similarly, JAL and JALR no longer use the ResultSrc=10 path of the single-cycle design; instead, a dedicated link control bit is propagated through the pipeline stages to signal that PC+4 should be written to the destination register at write-back. Finally, the control unit produces the same logical signals as before, but rather than being applied combinationally across the entire datapath in a single cycle, they are now registered alongside the instruction data and forwarded stage by stage through the pipeline.

5.6 Verification

The pipelined processor is verified with the same test program (finaltest_base.txt) and the same pass criterion as the single-cycle version: value 25 must be stored at memory address 100. The testbench loads the instruction memory via $readmemh("finaltest_pipeline.txt", imem, 0, 99), an identical copy of finaltest_base.txt, and detects completion when the PC locks onto the infinite-loop instruction at address 396. Simulation with Icarus Verilog produces the following output:

```
Score in mem[100]: 25
STATUS : ALL TESTS PASSED [score=25/25]
Cycles elapsed: 139
```

All 19 test groups pass. The cycle count printed by the testbench includes 15 cycles during which the testbench keeps running after the final loop is detected, so that the last store can commit; the program itself performs its final store at cycle 127. The cycles added with respect to the single-cycle version are introduced by the pipeline: two cycles are lost each time a branch is taken or a jump is executed, and one cycle is lost whenever a load instruction is immediately followed by an instruction that uses its result.

It is worth noting that the single-cycle version of this report organises the same program into 18 groups (G01–G18), where G01 covers both LUI and AUIPC together. In the pipelined testbench the two instructions are treated as separate test groups (T01 verifies the basic ADDI initialization and T02 verifies LUI and AUIPC) bringing the total to 19. The underlying test program and pass criterion remain identical in both versions: value 25 must be stored at memory address 100.

5.7 Module Summary

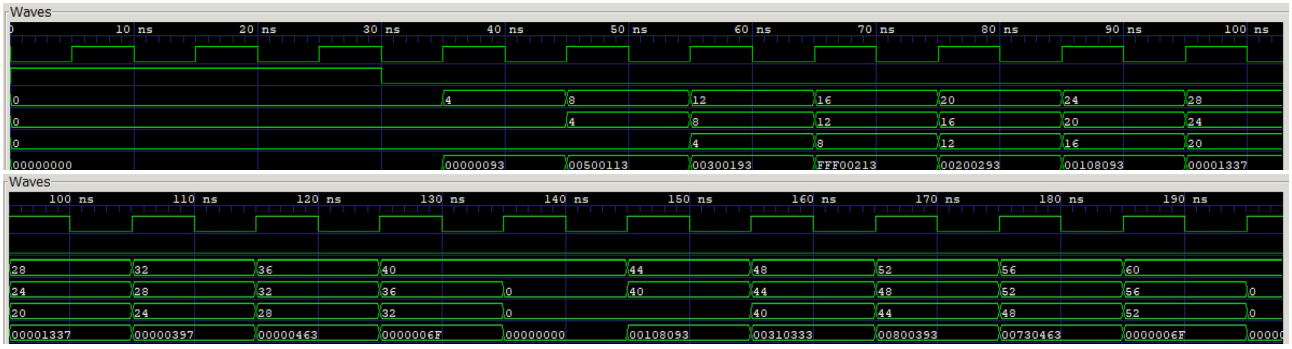
The pipelined implementation consists of the following Verilog modules:

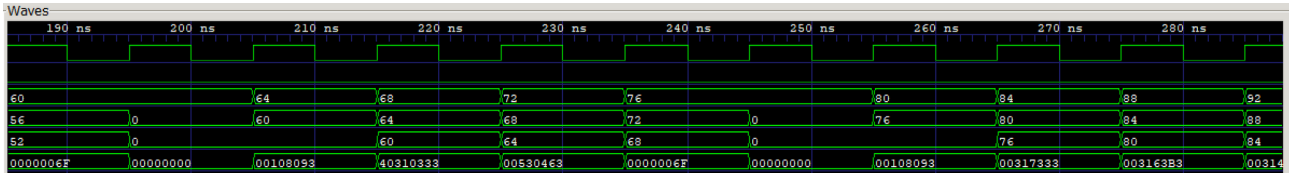
Module	Function
riscv_pipeline	Top-level. Contains all pipeline registers, PC, register file, forwarding logic, and hazard detection.
imm_gen	Equivalent to the extend module in the single-cycle version. Produces a sign-extended 32-bit immediate from the instruction encoding, supporting all five RV32I formats (I, S, B, J, U).
control_unit	Corresponds to the controller module of the single-cycle version (which internally instantiated maindec and aludec), here flattened into a single module.
alu	Extended from single-cycle: same base operations plus RV32M multiply/divide (alu_op widened to 5 bits)
branch_unit	New module, not present in the single-cycle version. Evaluates BEQ/BNE/BLT/BGE/BLTU/BGEU directly from the forwarded rs1/rs2 values in the EX stage, replacing the flag-based branch logic that was embedded in the single-cycle controller.
load_extend	Extracts and sign/zero-extends load data (LB/LH/LW/LBU/LHU) in the MEM stage. In the single-cycle version this logic was embedded directly inside the dmem module.

5.8 Waveform Analysis

5.8.1 Control Flow

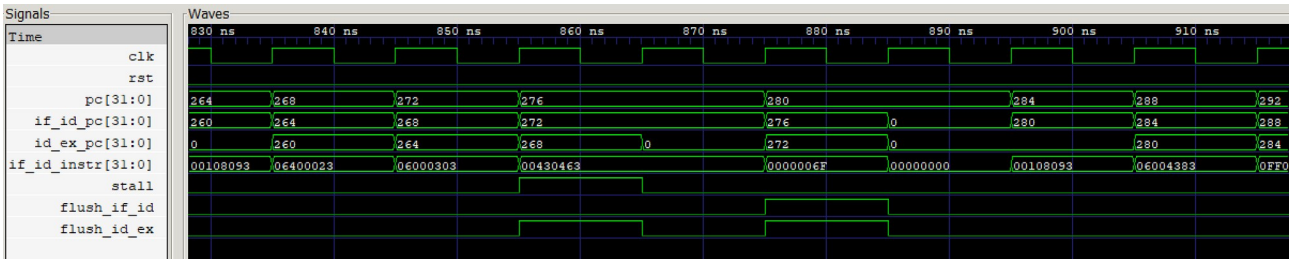
The first signals to inspect are those that track the instruction flow through the pipeline. The signal dut.pc shows the program counter at the IF stage and should advance by 4 each cycle under normal execution. The signals dut.if_id_pc and dut.id_ex_pc show the same PC value as it propagates through the first two pipeline registers, allowing the instruction to be tracked across stages. The signal dut.if_id_instr shows the raw instruction word currently in the IF/ID register and should be inspected in hexadecimal format to match against the test program. Jumps and taken branches are visible as discontinuities in the otherwise sequential PC progression.





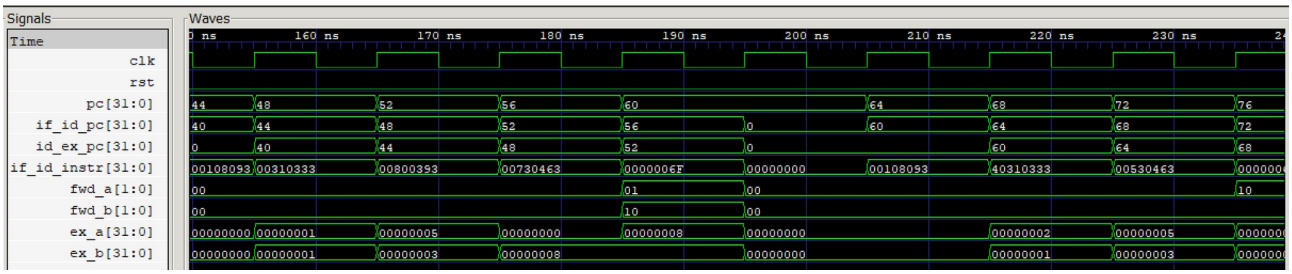
5.8.2 Hazard and Stall Signals

The hazard detection and flush signals reveal how the pipeline responds to data and control hazards. The signal dut.stall goes high for exactly one cycle whenever a load-use hazard is detected, during which the PC and IF/ID register are frozen and a bubble is injected into ID/EX. The signal dut.flush_if_id goes high for one cycle whenever a branch is taken or a jump is executed, invalidating the instruction that was fetched speculatively. The signal dut.flush_id_ex goes high both on taken branches and on load-use stalls.



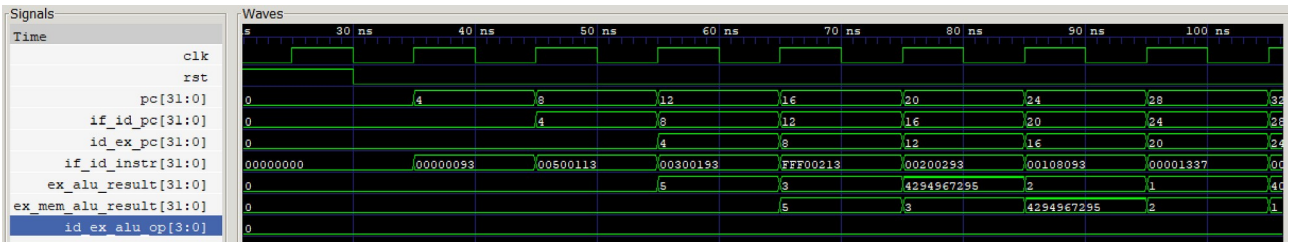
5.8.3 Forwarding

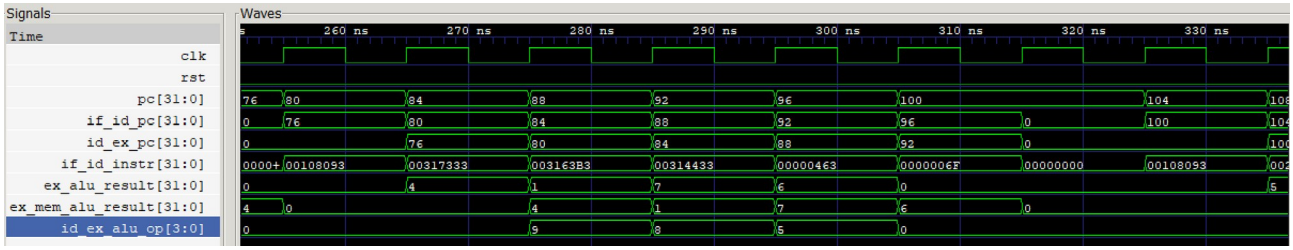
The forwarding unit signals show which data path is active on each cycle. The signal dut.fwd_a controls the first ALU operand: a value of 00 means no forwarding and the register file output is used directly, 01 means the value is forwarded from the MEM/WB stage, and 10 means it is forwarded from the EX/MEM stage. The signal dut.fwd_b follows the same encoding for the second ALU operand. The signals dut.ex_a and dut.ex_b show the actual values entering the ALU after forwarding is applied and are the most direct way to verify that the correct data reaches the execution stage.



5.8.4 ALU

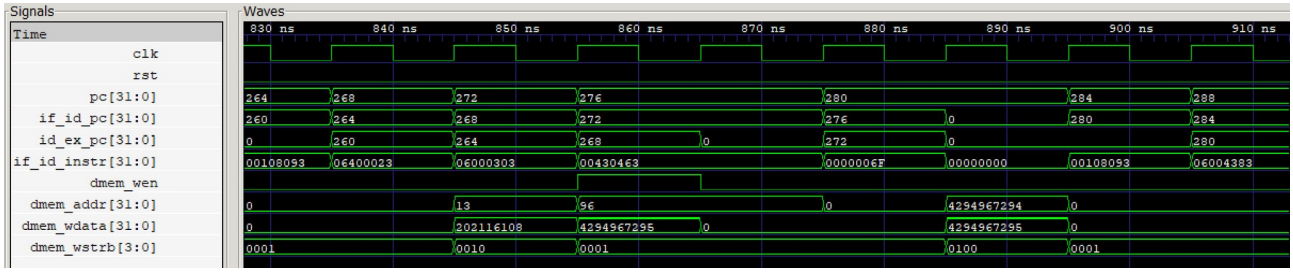
The signal dut.ex_alu_result shows the ALU output in the EX stage, and dut.ex_mem_alu_result shows the same value after it has been latched into the EX/MEM register. The signal dut.id_ex_alu_op shows the operation being performed, encoded as a 5-bit value (0=ADD, 1=SUB, 2=SLL, 3=SLT, 4=SLTU, 5=XOR, 6=SRL, 7=SRA, 8=OR, 9=AND).





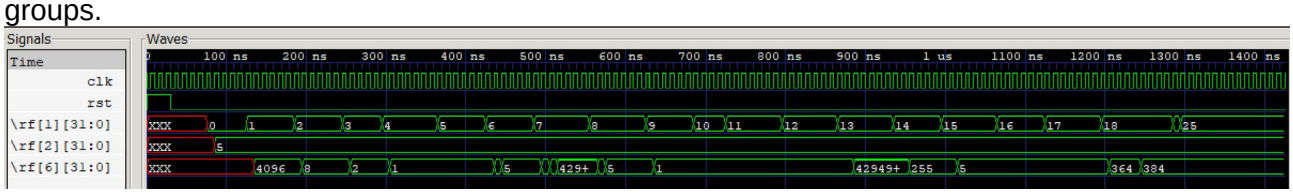
5.8.5 Data Memory

The signal dut.dmem_wen goes high whenever a store instruction is in the MEM stage. The signal dut.dmem_addr shows the byte address being accessed, and dut.dmem_wdata shows the data being written. The signal dut.dmem_wstrb shows the byte enable mask: 1111 for SW, 0011 or 1100 for SH, and 0001, 0010, 0100, or 1000 for SB depending on the byte offset. The signal dmem_rdata at the testbench level shows the data returned by the memory on a load.



5.8.6 Register File

The signal dut.rf[1] tracks register x1 (ra), which is used as the pass counter and should reach the value 25 at the end of the test program. The signal dut.rf[2] should remain constant at 5 throughout the simulation, and dut.rf[6] changes frequently as it is used as a scratch register across most test groups.



6. RV32M Extension: Multiply and Divide

6.1 Overview

Following the implementation of the pipelined processor, the design was further extended with the RV32M standard integer multiplication and division instruction set. The extension adds eight new instructions, all sharing the same R-type encoding as the base integer operations but identified by the unique funct7 value 0000001. No changes were required to the pipeline structure, the hazard detection unit, or the forwarding logic, since all RV32M instructions are register-to-register operations that follow the same five-stage datapath as any other R-type instruction.

6.2 Implemented Instructions

Instruction	Operation	Result
MUL	`rd = (rs1 × rs2)[31:0]`	Lower 32 bits of signed product
MULH	`rd = (rs1 × rs2)[63:32]`	Upper 32 bits, signed × signed
MULHSU	`rd = (rs1 × rs2)[63:32]`	Upper 32 bits, signed × unsigned

MULHU	`rd = (rs1 × rs2)[63:32]`	Upper 32 bits, unsigned × unsigned
DIV	`rd = rs1 / rs2`	Signed integer division
DIVU	`rd = rs1 / rs2`	Unsigned integer division
REM	`rd = rs1 % rs2`	Signed remainder
REMU	`rd = rs1 % rs2`	Unsigned remainder

6.3 Implementation Details

The changes to the Verilog source are confined to two modules and are minimal by design. In control_unit, the alu_op signal was widened from 4 to 5 bits to accommodate the eight new operation codes (10 through 17), and a new decoding branch was added inside the R-type case: when funct7 == 7'b0000001, the instruction is identified as RV32M and the eight variants are decoded from funct3 and mapped to their respective ALU operation codes. All other opcodes are unaffected. In alu, eight new case entries were added corresponding to the new operation codes. The four multiply variants compute a 64-bit intermediate product using Verilog functions that explicitly sign- or zero-extend the 32-bit operands before multiplying, then return either the lower or upper 32 bits of the result. The four divide and remainder variants use the Verilog signed division and modulo operators, with division by zero handled explicitly as mandated by the RISC-V specification: the quotient is the all-ones value and the remainder is the dividend. The signed overflow case, which occurs only when the most negative integer is divided by minus one, is not handled by dedicated logic; in simulation the Verilog operators already return the values required by the specification, namely the most negative integer as the quotient and zero as the remainder.

6.4 Verification

The RV32M extension is verified by a dedicated test program (finaltest_mul.txt) and a dedicated testbench (sim/tb_pipeline_rv32m.v), using the same pass-counter structure as the base test program. Register x1 is incremented by one on each pass and the final value is stored at memory address 200. The simulation produces the following output:

```
Score in mem[200]: 8 / 8
STATUS : ALL RV32M TESTS PASSED [score=8/8]
Cycles elapsed: 66
```

6.5 Test Group Summary

#	Instructions	What is verified	Pass condition
T01	`MUL`	5 × 3 = 15	x6 == 15
T02	`MULH`	high(signed(−1) × signed(3)) = −1	x6 == x4 (tp = −1)
T03	`MULHU`	high(0xFFFFFFFF × 3) = 2	x6 == x5 (t0 = 2)
T04	`MULHSU`	high(signed(−1) × unsigned(3)) = −1	x6 == x4 (tp = −1)
T05	`DIV`	15 / 3 = 5	x6 == x2 (sp = 5)
T06	`DIVU`	15 / 3 = 5	x6 == x2 (sp = 5)
T07	`REM`	17 % 3 = 2	x6 == x5 (t0 = 2)
T08	`REMU`	17 % 3 = 2	x6 == x5 (t0 = 2)

6.6 Waveform Analysis

6.6.1 Identifying RV32M Instructions

The most direct way to identify a multiply or divide instruction in the waveform is through the signal dut.id_ex_alu_op. For RV32M instructions this signal takes values from 10 to 17, which are outside the range used by the RV32I base set (0 to 9). The corresponding dut.if_id_instr will show a hex value with bits [31:25] equal to 0000001, which is the funct7 field that distinguishes RV32M from all other R-type instructions.

6.6.2 Multiply Results

For MUL (alu_op=10), dut.ex_alu_result shows the lower 32 bits of the product. For MULH, MULHSU, and MULHU (alu_op=11, 12, 13), it shows the upper 32 bits. To verify a result, compare dut.ex_alu_result against the expected value one cycle after the multiply instruction enters the EX stage. For example, MUL x6, x2, x3 with x2=5 and x3=3 should produce ex_alu_result=15.

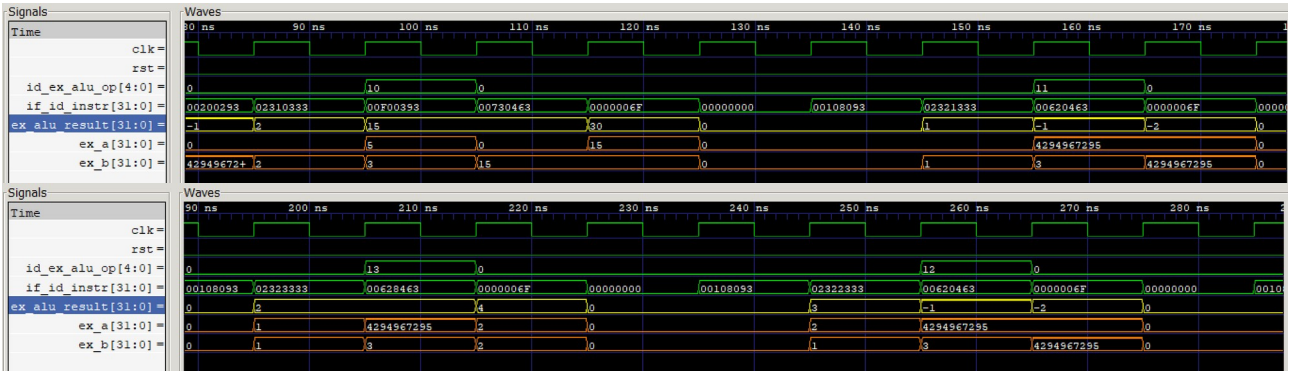
6.6.3 Divide and Remainder Results

For DIV, DIVU, REM, REMU (alu_op=14 to 17), dut.ex_alu_result shows the quotient or remainder. These are computed combinationally in a single cycle in the simulation model, so the result appears in the same cycle as the instruction in EX, exactly like any other ALU operation. The signals dut.ex_a and dut.ex_b show the dividend and divisor respectively after forwarding is applied.

6.6.4 Key Events to Identify

#	Instructions	What to look for
T01	`MUL`	id_ex_alu_op=10, ex_alu_result=15
T02	`MULH`	id_ex_alu_op=11, ex_alu_result=0xFFFFFFFF (−1)
T03	`MULHU`	id_ex_alu_op=13, ex_alu_result=2
T04	`MULHSU`	id_ex_alu_op=12, ex_alu_result=0xFFFFFFFF (−1)
T05	`DIV`	id_ex_alu_op=14, ex_a=15, ex_b=3, ex_alu_result=5
T06	`DIVU`	id_ex_alu_op=15, ex_alu_result=5
T07	`REM`	id_ex_alu_op=16, ex_a=17, ex_b=3, ex_alu_result=2
T08	`REMU`	id_ex_alu_op=17, ex_alu_result=2

The waveform uses a consistent colour scheme throughout. The single-bit control signals clk and rst, together with the bus signals id_ex_alu_op[4:0] and if_id_instr[31:0], are all shown in green. The signal ex_alu_result[31:0] is shown in yellow. The operand signals ex_a[31:0] and ex_b[31:0] are shown in orange. In all cases the decoded value is displayed in white text inside the coloured band. Transitions between values appear as crossing lines at the moment the signal changes, which in this waveform always coincides with a rising clock edge.





7. Performance Analysis

7.1 Cycles Per Instruction

In the single-cycle implementation, the CPI is exactly 1 by construction: every instruction, regardless of type, occupies the processor for precisely one clock cycle. This simplicity comes at a cost, however, since the clock period must be long enough to accommodate the slowest possible instruction from start to finish within a single cycle.

The pipelined implementation aims for a CPI close to 1 under ideal conditions, but real programs introduce penalties. A taken branch or any jump flushes two instructions already in flight, costing two extra cycles. A load instruction immediately followed by an instruction that uses its result triggers a one-cycle stall. The test program used for verification executes 78 instructions, as measured on the single-cycle core, where each instruction takes exactly one cycle. The pipelined core performs the final store of the same program at cycle 127; excluding the four cycles needed to fill the pipeline, this corresponds to approximately 1.6 cycles per instruction. The value is high because every test group ends with a taken branch, so the test program represents a worst case for the pipeline rather than typical code.

7.2 Critical Path

In the single-cycle design, the critical path is the load instruction, which must traverse the program counter, instruction memory, register file read, ALU adder, data memory, and the write-back mux all within a single clock period. This long combinational chain sets an upper bound on the clock frequency that applies to every instruction, even the simplest arithmetic ones.

The pipeline eliminates this constraint by distributing the work across five stages. Each stage now only needs to complete its own portion of the work before the next clock edge, so the clock period is determined by the slowest individual stage rather than the slowest complete instruction. In practice the EX stage, which contains the ALU, and the MEM stage, which accesses data memory, tend to be the bottlenecks.

7.3 Single-Cycle vs Pipelined: Summary

Metric	Single-cycle (this work)	5-stage pipeline
CPI	1 (fixed)	≈ 1.6 measured (78 instructions, final store at cycle 127)
Clock frequency	Limited by full load path	Limited by single-stage path
Hazard handling	Not needed	Forwarding, stall, flush

Branch penalty	None	2 cycles per taken branch or jump
Load-use penalty	None	1 cycle per load-use pair
Design complexity	Low	Higher

8. Conclusions and Future Work

8.1 Summary

This project developed a RISC-V processor in three stages, each building on the previous one. The single-cycle implementation established a correct and verifiable baseline covering all six RV32I instruction formats, all R-type and I-type ALU operations, all six branch conditions, sub-word memory access, and both JAL and JALR jumps. Verification through a self-checking testbench confirmed correctness by storing value 25 at address 100 after all 18 test groups passed.

The second stage introduced a five-stage pipeline (IF, ID, EX, MEM, WB) with full data forwarding to handle back-to-back register dependencies, load-use hazard detection with one-cycle stalls, and a predict-not-taken branch strategy that flushes two pipeline stages on misprediction. The same test program was reused without modification, confirming that the pipelined design produces identical results to the single-cycle version while reaching the final store at cycle 127.

The third stage added the RV32M integer multiply and divide extension to the pipelined processor, introducing eight new instructions (MUL, MULH, MULHSU, MULHU, DIV, DIVU, REM, REMU) with changes confined to the control unit and ALU. A dedicated test program verified all eight instructions, achieving a score of 8 out of 8.

8.2 Possible Improvements

While the design is fully functional and verified, several extensions would make it more complete. Synthesising the design on a target FPGA would provide concrete timing data and resource utilisation figures, and would allow execution of real compiled software rather than hand-assembled test programs. Adding exception and interrupt handling would bring the processor closer to a real embedded core. Support for the RV32F or RV32D floating-point extensions would significantly broaden the range of workloads the processor can handle. Finally, exploring more advanced branch prediction strategies, such as a branch history table, would reduce the two-cycle penalty currently paid on every taken branch.

8.3 Use of AI Tools

Parts of this project were developed with the assistance of Claude (Anthropic). AI assistance was specifically used to improve inline code commenting and documentation, to refine and polish the written sections of this report, and to support the debugging process during simulation, in particular for the pipeline hazard logic and the RV32M multiply implementation. The use of AI tools was limited to an assistive role and did not replace the author's own understanding or judgement at any stage of the project.

9. Appendix

9.1 File Structure

9.1.1 Single-Cycle Implementation

```
rtl/riscv_single_cycle.v        — single-cycle processor (all modules)
sim/tb_single_cycle.v           — self-checking testbench
```

```
programs/finaltest_base.txt      — 100-instruction test program
riscvsingle1.vcd                 — waveform output (generated at simulation time)
```

9.1.2 Pipelined Implementation

```
rtl/riscv_pipeline.v             — pipelined processor with RV32M extension
sim/tb_pipeline.v                — RV32I testbench
sim/tb_pipeline_rv32m.v          — RV32M testbench
programs/finaltest_pipeline.txt  — 100-instruction RV32I test program
programs/finaltest_mul.txt       — 44-instruction RV32M test program
riscv_pipeline.vcd               — RV32I waveform output
riscv_pipeline_m.vcd             — RV32M waveform output
```

9.2 Simulation Instructions

```
# Single-cycle: compile and run
iverilog -g2012 -o sim_sc rtl/riscv_single_cycle.v sim/tb_single_cycle.v
(cd programs && vvp ../sim_sc)
gtkwave programs/riscvsingle1.vcd

# Pipeline RV32I: compile and run
iverilog -g2012 -o sim_i rtl/riscv_pipeline.v sim/tb_pipeline.v
(cd programs && vvp ../sim_i)
gtkwave programs/riscv_pipeline.vcd

# Pipeline RV32M: compile and run
iverilog -g2012 -o sim_m rtl/riscv_pipeline.v sim/tb_pipeline_rv32m.v
(cd programs && vvp ../sim_m)
gtkwave programs/riscv_pipeline_m.vcd
```